Regex

$regex : Provides regular expression capabilities for pattern matching strings in queries.

To use $regex, use one of the following syntax:
{ <field>: { $regex: /pattern/} }

case 1: Find all names where "test" occurs as a substring or as a separate word.

```
db.lju.find({name:{$regex:/test/}})
or
db.lju.find({name:{$regex:"test"}})
```

case 2: To have a case-insensitive matching we can append /i indentifier after RE

```
db.lju.find({name:{$regex:/test/i}})
```

This will return documents if name is stored in collection as "Test","tEST", "teST","Testing" etc.

case 3: To match a string beginning with "test" only.

```
db.lju.find({name:{$regex:/^test/}})
```

case 4: To end with "test" only.

```
db.lju.find({name:{$regex:/test$/}})
```

case 5: To end with digit only.

```
db.lju.find({name:{$regex:/[0-9]$/}})
```

case 6: To start with digit only.

```
db.lju.find({name:{$regex:/^[0-9]/}})
or
db.lju.find({name:{$regex:/^\d/}})
```

case 7: To accept only digits, nothing else. Not even blank.

```
db.lju.find({name:{$regex:/^[0-9]+$/}})
```

case 8: To accept only with empty string also.

```
db.lju.find({name:{$regex:/^[0-9]*$/}})
```

case 9: To match having a name of 3-10 letters only.

```
db.lju.find({name:{$regex:/^[A-Za-z]{3,10}$/}})
```

Letters only is mentioned in question so we have added ^ and $.

Note:
If we include \w instead of this [A-Za-z], then it may allow digits & underscore also.
If it is asked in question that it **allowed only letters** then **use [A-Za-z]**

Note: all patterns can be matched with string only. If there is one field age and we have inserted all int values then RegEx can not be compared with it.

Example

_id	name	age	city	marks	status
1	Amit	18	Ahmedabad	75	A
2	Ankit123	20	Surat	82	A
3	Riya	21	Rajkot	68	B
4	TestUser	22	Vadodara	90	A
5	UserTest	19	Ahmedabad	71	B
6	Testing	20	Surat	84	A
7	ABC123	18	Rajkot	65	B
8	123ABC	23	Ahmedabad	88	A
9	98765	22	Bhavnagar	59	C
10	Anjali	20	Surat	80	A

Question	MongoDB Query
1. Display students whose name contains "Test".	db.student.find({name:{$regex:/Test/}})
2. Display students whose name contains "test" (case-insensitive).	db.student.find({name:{$regex:/test/i}})
3. Display students whose name starts with "A".	db.student.find({name:{$regex:/^A/}})
4. Display students whose name ends with "Test".	db.student.find({name:{$regex:/Test$/}})
5. Display students whose name starts with a digit.	db.student.find({name:{$regex:/^[0-9]/}})
6. Display students whose name ends with a digit.	db.student.find({name:{$regex:/[0-9]$/}})

Question	MongoDB Query
7. Display students whose name contains only digits.	db.student.find({name:{$regex:/^[0-9]+$/}})
8. Display students whose name contains only letters.	db.student.find({name:{$regex:/^[A-Za-z]+$/}})
9. Display students whose name starts with a letter and ends with a digit.	db.student.find({name:{$regex:/^[A-Za-z].*[0-9]$/}})
10. Display students whose name starts with "A" or "R".	db.student.find({name:{$regex:/^(A|R)/}})
Display students whose name contains **at least one digit**.	db.student.find({name:{$regex:/[0-9]/}})
Display students whose name starts with **two uppercase letters**.	db.student.find({name:{$regex:/^[A-Z]{2}/}})
Display students whose name starts with a letter and has **exactly three digits at the end**.	db.student.find({name:{$regex:/^[A-Za-z]+[0-9]{3}$/}})

Indexing in MongoDB

- MongoDB uses indexing in order to make the query processing more efficient. If there is no indexing, then the MongoDB must scan all documents in the collection and retrieve only those documents that match the query.
- Indexes are special data structures that stores some information related to the documents such that it becomes easy for MongoDB to find the right data file. The indexes are order by the value of the field specified in the index.

When not to create indexing:

- **When the collection is small**, as scanning the entire collection is often faster than maintaining an index.
- **When the collection is updated frequently** (frequent insert, update, or delete operations), because indexes must also be updated, which can reduce write performance.
- **When queries do not repeatedly use the same fields**, as the index will rarely be used and may not improve performance.
- **When too many indexes are created**, as they consume additional storage space and slow down insert, update, and delete operations.

If your application is repeatedly running queries on the same fields, you can create an index on those fields to improve performance. For example, consider the following scenarios:

Scenario	Index Type
A human resources department often needs to look up employees by employee ID. You can create an index on the employee ID field to improve query performance.	**Single Field Index**
A store manager often needs to look up inventory items by name and quantity to determine which items are low stock. You can create a single index on both the item and quantity fields to improve query performance.	**Compound Index**

- Indexes are special data structures that store a small portion of the collection's data set in an easy-to-traverse form.
- MongoDB indexes use a B-tree data structure.
- The index stores the value of a specific field or set of fields, ordered by the value of the field.

Default Index

MongoDB creates a unique index on the _id field during the creation of a collection. The _id index prevents clients from inserting two documents with the same value for the _id field. You cannot drop this index.

Index Names

The default name for an index is the combination of the **index keys** and each key's **direction** in the index **(1 or -1)** with **underscores** as a separator.

For example, an index created on { item : 1, quantity: -1 } has the name item_1_quantity_-1.

explain() method

Provides information on the query plan for the db.collection.find() method.
The explain() method has the following form:
db.collection.find().explain()
The following example runs cursor.explain() in "executionStats" verbosity mode to return the query planning and execution information for the specified db.collection.find() operation:

db.students.find({age:{$gt:15}}).explain("executionStats")

Explain(executionStats) is a method that you can apply to simple queries or to cursors to investigate the query execution plan. The execution plan is how MongoDB resolves a query. Looking at all the information returned by explain():

- totalDocsExamined: how many documents were examined
- nReturned:how many documents were returned
- stage: Inside the winningPlan -> queryPlan -> stage is defined. It is used to defined the stage of input scanning.
    - COLLSCAN for a collection scan
    - IXSCAN for scanning index keys

- ✓ The query was a simple filter operation on the age field (age < 15).
- ✓ Since no suitable index was available, MongoDB had to perform a full collection scan (COLLSCAN), examining all 6 documents in the people collection.
- ✓ Out of the 6 documents examined, 3 matched the filter criteria and were returned.
- ✓ The query execution was successful and relatively quick, taking 10 milliseconds to complete.

❖ createIndex

MongoDB provides a method called createIndex() that allows user to create an index.

Syntax: db.**collection_name**.createIndex({KEY:1})

MongoDB only creates the index if an index of the same specification does not exist.
The key determines the field on the basis of which you want to create an index and 1 (or -1) determines the order in which these indexes will be arranged

1 for ascending order and -1 for descending order

1. Create an Index on a Single Field

We can create an index on any one of the fields of collection.

Example:

```
db.table1.createIndex({age:1})
```

This will create an index named "age_1".

2. Compound indexing

Compound indexes are indexes that contain references to multiple fields. Compound indexes improve performance for queries on exactly the fields in the index or fields in the index prefix.
To create a compound index, use the db.collection.createIndex() method:
db.<collection>.createIndex({<field1>:<sortOrder>,<field2>:<sortOrder>,...,<fieldN>: <sortOrder>})

Example:

```
db.table1.createIndex({age:1,name:-1})
```

An index created on { age : 1, name: -1 } has the name age_1_name_-1.

In this example:

- The index on age is ascending (1).
- The index on name is descending (-1).

The created index supports queries that applies on:

- Both age and name fields.
- Only the age field, because age is a prefix of the compound index.

For example, the index supports these queries:

```
db.students.find( { name: "Ali", age: 36 } )
db.students.find( { age: 26 } )
```

The index does not support queries on only the name field, because name is not part of the index prefix. For example, the index does not support this query:

```
db.students.find( { name: "Bob" } )
```

❖ getIndexes()

To confirm that the index was created, use below command:

```
db.collection.getIndexes()
```

Output:

[{ v: 2, key: { _id: 1 }, name: '_id_' },{ v: 2, key: { age: 1 }, name: 'age_1' }]

❖ dropIndex()/dropIndexes()

dropIndex() : Drops or removes the specified index from a collection.

```
db.collection.dropIndex(index)
```

dropIndexes() : It is used to drop all indexes except the _id index from a collection.

```
db.collection.dropIndexes()
```

Drop a specified index from a collection. To specify the index, you can pass the method either:

- **The index specification document**
  db.collection.dropIndexes({ a: 1, b: 1 })
- **The index name:**
  db.collection.dropIndexes("a_1_b_1")
- **Drop specified indexes from a collection. To specify multiple indexes to drop, pass the method an array of index names:**
  db.collection.dropIndexes(["a_1_b_1", "a_1", "a_1__id_-1"])

If the array of index names includes a non-existent index, the method errors without dropping any of the specified indexes

To get the names of the indexes, use the db.collection.getIndexes() method.

Let's understand the concept with following example.

Suppose, we have a collection named student and we want to apply query to find students who **have age greater than 12**.

Student Collection

_id	Name	Age
1	ABC	10
2	XYZ	12
3	ABC	15
4	PQR	13
5	XYZ	15
6	ABC	8

- **Find students without creating an index.**

db.student.find({age:{$gt:12}}).explain("executionStats")
It will examine 6 documents and return 3 documents by performing COLLSCAN.
totalDocsExamined: 6
nReturned:3
stage: COLLSCAN

Single Indexing Example

- **Create an index on the age field to improve performance for those queries:**

db.students.createIndex({ age: 1 })
index name: age_1

- **Now, find the students with age greater than 12**

db.student.find({age:{$gt:12}}).explain("executionStats")
It will examine 3 documents and return 3 documents by performing IXSCAN.
totalDocsExamined: 3
nReturned: 3
stage: IXSCAN

Compound indexing Example

Create an index on the age and name fields to improve performance.(Compound Indexing)

Note: Before creating any other indexing on same fields. Drop the created indexing

db.students.createIndex({ age: 1,name:-1 })
index name: age_1_name_-1

So, the order of documents in this index would be:

_id: 6	age: 8	name: ABC
_id: 1	age: 10	name: ABC
_id: 2	age: 12	name: XYZ
_id: 4	age: 13	name: PQR
_id: 5	age: 15	name: XYZ
_id: 3	age: 15	name: ABC

- So here on name field indexing is applied based on age field.

- In this example we have two students with age 15. For age field where value is 15, the name field values are arranged in descending order.
- "name" field indexing is depending on the age field. So, if we apply query on name field then it will perform the collscan.

Now, consider below three scenarios

1. **Display student with name "ABC" and age 15**

db.student.find({age:15,name:"ABC"}).explain("executionStats")
It will examine 1 documents and return 1 documents by performing IXSCAN.
totalDocsExamined: 1
nReturned: 1
stage: IXSCAN

2. **Display students whose age is greater than 12**

db.student.find({age:{$gt:12}).explain("executionStats")
It will examine 3 documents and return 3 documents by performing IXSCAN.
totalDocsExamined: 3
nReturned: 3
stage: IXSCAN

3. **Display students whose name is "ABC"**

db.student.find({name:"ABC"}).explain("executionStats")
It will examine 6 documents and return 3 documents by performing COLLSCAN.
totalDocsExamined: 6
nReturned: 3
stage: COLLSCAN

❖ Partial Indexing

Partial Indexing is a feature in MongoDB that allows you to create an index on a subset of documents within a collection, rather than on the entire collection. This can be particularly useful for optimizing performance by reducing the size of the index and improving the speed of certain queries.

Key Concepts:

- **Subset of Documents**: A partial index only includes documents that meet a specified filter condition.

- **Performance Improvement**: By indexing only relevant documents, you can save space and potentially improve query performance, especially if the filter condition matches the majority of your queries.
- **Use Cases**: Commonly used when you have sparse data or when only certain documents in a collection are frequently queried.

The partialFilterExpression option accepts a document that specifies the filter condition using:

- equality expressions (i.e. field: value or using the $eq operator),
- $exists: true expression,
- $gt, $gte, $lt, $lte expressions,
- $type expressions,
- $and operator,
- $or operator,
- $in operator

How to Create a Partial Index

To create a partial index, you use the **partialFilterExpression** option along with the **createIndex** command.

Example:

Assume you have a collection called students, and you often query for students who have an age field greater than 15. Instead of indexing all documents, you can create a partial index that only includes documents where age is greater than 15.

```
db.students.createIndex(
  { age: 1 },
  { partialFilterExpression: { age: { $gt: 15 } } }
)
```

How It Works:

- **Index Definition**: The index { age: 1 } is created, but only for documents where the age field is greater than 15.
- **Smaller Index**: The index size is reduced because it excludes documents that don't meet the condition (i.e., documents where age is 15 or less).
- **Optimized Queries**: Queries like db.students.find({ age: { $gt: 15 } }) can use this index effectively, improving performance.

Example:

Consider student collection with 6 documents

- **Create index on age where age is greater than 15**

```
db.student.createIndex({age:1},{partialFilterExpression:{age:{$gt:15}}})
```

- **Display student/s whose age is 16.**

```
db.student.find({age:16}).explain("executionStats")
```

Output: stage: 'IXSCAN',
nReturned: 1,
docsExamined: 1

Suppose we have only one document available with age value 16.

- **Display student/s whose age is 13.**

```
db.student.find({age:13}).explain("executionStats")
```

Output: stage: 'COLLSCAN',
nReturned: 1,
docsExamined: 6

Suppose we have only one document available with age value 13.

We have applied index on age field where age is greater than 15. we are looking for age 13 which is less than 15. Then here it will perform the COLLSCAN.

- **Display student/s whose age is 17.**

```
db.student.find({age:17}).explain("executionStats")
```

Output: stage: 'IXSCAN',
nReturned: 2,
docsExamined: 2

Suppose we have only 2 documents available with age value 17.

We have applied index on age field where age is greater than 15. we are looking for age 17 which is greater than 15. Then here it will perform the IXSCAN.

Winning Plan

When multiple indexes are available for a query, MongoDB's **query optimizer** evaluates different execution plans and selects the most efficient one. The selected execution plan is called the **Winning Plan**, while the other possible execution plans are called **Rejected Plans**.
The winning and rejected plans can be viewed using the explain("executionStats") method.

Example:

Create two indexes on the same field:

```
db.students.createIndex({age:1})
db.students.createIndex({age:1,name:1})
```

Run the following query:

```
db.students.find({age:19}).explain("executionStats")
```

MongoDB compares both indexes and selects the most efficient one as the **Winning Plan**. The remaining index is displayed under **Rejected Plans** in the explain() output.

Note:

- **Winning Plan:** The execution plan chosen by MongoDB for the query.
- **Rejected Plans:** Other execution plans considered but not selected because they are less efficient.

Example:

Consider a collection student having documents like this:

[
{_id:123433,name: "DDD",age:32},
{_id:123434,name: "BBB",age:20},
{_id:123435,name: "AAA",age:10},
]

Do as directed:

(1) Create an index & fire a command to retrieve a document having age>15 and name is "BBB". Stats must return values nReturned=1, docExamined=1, stage="IXSCAN". Perform required indexing.

(2) Create an index on subset of a collection having age>30. Also write a command to get a stats "IXSCAN" for age>30.

Solution:

1) **db.student.createIndex({age:1,name:1})**
   db.student.find({age:{$gt:15},name:'DDD'}).explain('executionStats')

2) **db.student.createIndex({age:1},{partialFilterExpression:{age:{$gt:30}}})**
   db.student.find({age:{$gt:30}}).explain('executionStats')

Example:

Create student collection and perform the task as ask below.

```
[
{ _id:1, name:"ABC", city:"Ahmedabad", age:18, marks:75 },
{ _id:2, name:"XYZ", city:"Surat", age:20, marks:82 },
{ _id:3, name:"PQR", city:"Ahmedabad", age:19, marks:68 },
{ _id:4, name:"ABC", city:"Rajkot", age:21, marks:90 },
{ _id:5, name:"XYZ", city:"Ahmedabad", age:20, marks:85 },
{ _id:6, name:"LMN", city:"Surat", age:18, marks:60 }
]
```

Question 1: Simple Index

Create an index on the age field and retrieve all students whose age is 20. Verify the execution statistics.

```
db.student.createIndex({age:1})

db.student.find({age:20}).explain("executionStats")
```

Expected Output

stage : IXSCAN
nReturned : 2
totalDocsExamined : 2

Question 2: Compound Index

Drop the existing index. Create a compound index on age and name. Retrieve the student whose age is 20 and name is "XYZ". Verify the execution statistics.

```
db.student.dropIndex("age_1")
db.student.createIndex({age:1,name:1})
db.student.find({age:20,name:"XYZ"}).explain("executionStats")
```

Expected Output

stage : IXSCAN
nReturned : 2
totalDocsExamined : 2

Question 3:

Using the same compound index, retrieve all students whose name is "ABC". Verify the execution statistics.

```
db.student.find({name:"ABC"}).explain("executionStats")
```

Expected Output

stage : COLLSCAN
nReturned : 2
totalDocsExamined : 6

Question 4: Partial Index

Drop the existing compound index. Create a partial index on the marks field for students having marks greater than 80. Retrieve all students whose marks are greater than 80 and verify the execution statistics.

```
db.student.dropIndex("age_1_name_1")
db.student.createIndex(
  {marks:1},
  {partialFilterExpression:{marks:{$gt:80}}}
)
db.student.find({marks:{$gt:80}}).explain("executionStats")
```

Expected Output

stage : IXSCAN
nReturned : 3
totalDocsExamined : 3
(Students with marks **82, 85, and 90**.)

Question 5:

Using the same partial index, retrieve all students whose marks are greater than 70. Verify the execution statistics.

```
db.student.find({marks:{$gt:70}}).explain("executionStats")
```

Expected Output

stage : COLLSCAN
nReturned : 4
totalDocsExamined : 6

Mongoose

Elegant MongoDB object modeling for Node.js.

Mongoose is an ODM (Object Data Modeling) library for MongoDB. While you don't need to use an Object Data Modeling (ODM) or Object Relational Mapping (ORM) tool to have a great experience with MongoDB. Many Node.js developers choose to work with Mongoose to help with data modeling, schema enforcement, model validation, and general data manipulation. And Mongoose makes these tasks effortless.

Why Mongoose?

By default, MongoDB has a flexible data model. This makes MongoDB databases very easy to alter and update in the future. Mongoose forces a semi-rigid schema from the beginning. With Mongoose, developers must define a Schema and Model.

First be sure you have MongoDB and Node.js installed.

Install Mongoose from the command line using npm:

```
npm install mongoose
```

The first thing we need to do is include mongoose in our project and open a connection to the test database on our locally running instance of MongoDB.

```
const mg = require('mongoose');
mg.connect("mongodb://127.0.0.1:27017/test").then(()=>{console.log("success")}).catch((err)=>{console.error(err)});
```

What is a Schema?

A **Schema** defines the structure of documents in a MongoDB collection. It specifies the **field names, data types, and validation rules** for the documents.

Mongoose Data Types

Data Type	Purpose
String	Stores text
Number	Stores numbers

Data Type	Purpose
Boolean	Stores true or false
Date	Stores date and time
ObjectId	Stores document reference
Array	Stores multiple values

Mongoose Schema Validation

Schema Validation checks whether the data follows the rules defined in the schema before it is stored in MongoDB.

Why Use Validation?

- Ensures only valid data is stored.
- Prevents incorrect or incomplete data.
- Maintains data consistency.

When is Validation Performed?

- Before inserting a document.
- Before saving a document using save().
- During update operations only if runValidators:true is used.

What Happens if Validation Fails?

- The document is **not saved**.
- Mongoose throws a **ValidationError**.

Validators and Schema Options

Validator / Option	Purpose
Required	Makes a field mandatory
Unique	Prevents duplicate values

Validator / Option	Purpose
Default	Assigns a default value
min, max	Sets minimum and maximum values
minlength, maxlength	Sets minimum and maximum string length
Enum	Allows only specified values
Match	Validates using a regular expression
Trim	Removes leading and trailing spaces
Lowercase	Converts text to lowercase
Uppercase	Converts text to uppercase
Validate	Defines a custom validation function
Strict	Allows or restricts fields outside the schema

strict Option

The **strict** option controls whether fields that are **not defined in the schema** are stored.

strict:true (Default)	strict:false
Stores only schema fields	Stores schema fields and extra fields
Extra fields are ignored	Extra fields are also saved

Syntax understanding

```
const mySchema = new mg.Schema(
{
   name:{
      type:String,
      required:true
   },
   surname:String,
```

```
  age:Number,
  active:Boolean,
  date:{
    type:Date,
    default:new Date()
  }
},
{
  strict:false
});
```

What is a model?

Models take your schema and apply it to each document in its collection.

Models are responsible for all document interactions like creating, reading, updating, and deleting (CRUD).

An important note: the first argument passed to the model should be the singular form of your collection name. Mongoose automatically changes this to the plural form, transforms it to lowercase, and uses that for the database collection name.

```
const person=new mg.model("person",mySchema)
```

it will automatically converted to “people” by mongoose

```
mg.pluralize(null) // to add collection as we have mentioned
```

Inserting data

Now that we have our first model and schema set up, we can start inserting data into our database.

Back in the file, let’s insert a new data.

```
const createDoc=async()=>
  {
   try{
     const personData=new person(
       {
         name:"def",
         Surname:"test",
         age:31,
         active:true,
         email:"abc@gmail.com"
```

```
        }
      )
   const result=await personData.save();
}
}
createDoc()
```

we create a new object and then use the save() method to insert it into our MongoDB database.

Await and Async

If you are using async functions, you can use the await operator on a Promise to pause further execution until the Promise reaches either the Fulfilled or Rejected state and returns. Since the await operator waits for the resolution of the Promise, you can use it in place of Promise chaining to sequentially execute your logic.

The async and await keywords in JavaScript are used to handle asynchronous operations in a more readable and straightforward way compared to traditional methods like callbacks and Promises. Here's an explanation of their use and benefits:

async Keyword

The async keyword is used to define an asynchronous function. When a function is declared as async, it implicitly returns a Promise. This means you can use the await keyword inside this function to pause its execution until the awaited Promise is resolved or rejected.

await Keyword

The await keyword can only be used inside an async function. It pauses the execution of the async function and waits for the resolution (or rejection) of a Promise. Once the Promise is resolved, the function resumes execution, and the resolved value of the Promise is returned. If the Promise is rejected, the await expression throws the rejected value.

Benefits of async and await

1. **Readability**: The syntax of async and await makes asynchronous code look more like synchronous code, which is easier to read and understand.
2. **Error Handling**: Handling errors with async/await is more straightforward using try/catch blocks compared to handling errors with Promises.
3. **Avoiding Callback Hell**: async/await helps to avoid deeply nested callbacks, making the code cleaner and more maintainable.

Example1:

Write a node.js script to enter one document to the collection after establishing a connection using mongoose. Raise exception and catch it, if any problem occurs. Print a proper message for error handling.

```
const mg=require("mongoose")
mg.connect("mongodb://127.0.0.1:27017/lju").then(()=>{console.log("success")}).catch((err)=
>{console.error(err)});

//mg.pluralize(null)  The mg.pluralize(null) line is commented out. If uncommented, it would disable Mongoose's automatic pluralization of the collection name. By default, Mongoose pluralizes the model name (e.g., person to people). Uncommenting this line would ensure that the collection name remains person instead of being pluralized to people.


const mySchema=new mg.Schema(
   {
      name:{
         type:String,
         required:true
      },
      Surname:String,
      age:Number,
      active:Boolean,
      date:{
         type:Date,
         default:new Date().toLocaleDateString()
      }
   }
);
const person=new mg.model("person",mySchema)
const createDoc=async()=>
{
   try{
      const personData=new person({
         name:"test",
         Surname:"XYZ",
         age:3,
```

```
        active:true
      })
      const result=await personData.save(); //for single data record
      console.log(result);
   }
   catch(err)
   {
      console.log("Error Occured" + err);
   }
}
createDoc();
```

Explanation of Example 1

Step 1: Importing Mongoose and Connecting to MongoDB

- ✓ **const mg = require("mongoose"); :** This imports the Mongoose library, which is an Object Data Modeling (ODM) library for MongoDB and Node.js.
- ✓ **mg.connect("mongodb://127.0.0.1:27017/lju"):** This line establishes a connection to a MongoDB database named **lju** running on the local machine (localhost) at the default port 27017.
- ✓ **.then(...):** If the connection is successful, it logs "success" to the console.
- ✓ **.catch(...):** If there is an error during connection, it logs the error to the console.

Step 2: Defining a Schema

- ✓ **const mySchema = new mg.Schema({ ... });:** This defines a new schema for a collection. A schema defines the structure of the documents within a collection.
- ✓ The schema includes the following fields:
  - name: A required string field.
  - Surname: An optional string field.
  - age: An optional number field.
  - active: An optional boolean field.
  - date: A date field with a default value set to the current date, formatted as a locale date string.
- ✓ The default value for the date field is set using new Date().toLocaleDateString(), which will set the date as a string formatted according to the local date format.

Step 3: Creating a Model

- ✓ **const person = new mg.model("person", mySchema);:** This creates a Mongoose model named "person" using the defined schema (mySchema).

✓ The model is used to interact with the person collection in the MongoDB database.

Step 4: Creating and Saving a Document

- ✓ **const createDoc = async () => { ... };:** This defines an asynchronous function createDoc to create and save a document in the MongoDB collection.
- ✓ **const personData = new person({ ... });:** This creates a new instance of the person model with the specified data:
  - name: "test"
  - Surname: "XYZ"
  - age: 3
  - active: true
- ✓ **const result = await personData.save();:** This saves the document to the database and waits for the operation to complete.
- ✓ **console.log(result);:** If the document is successfully saved, it logs the resulting document to the console.
- ✓ **catch (err) { ... }:** If an error occurs during the save operation, it logs the error to the console.

Summary

- ✓ The code imports Mongoose and connects to a local MongoDB instance.
- ✓ It defines a schema for the person collection with fields for name, Surname, age, active, and date.
- ✓ It creates a model for the person collection based on the schema.
- ✓ It defines an asynchronous function to create a new document and save it to the database.
- ✓ If the document is successfully saved, it logs the document to the console; otherwise, it logs an error.

In Mongoose and MongoDB, __v is a special field that stands for **version**. It is used to track the version of a document within a collection. Here's a detailed explanation:

Purpose of __v

id: ObjectId('66c1209c8ba3569ea4cd15f7')
name: "test"
age: 25
email: "test@example.com"
gender: "MALE"
__v: 0

Version Tracking:

1. The __v field is automatically managed by Mongoose to handle concurrency control. Each time a document is updated, its version number (__v) is incremented.
2. This helps in managing optimistic concurrency control. If two processes attempt to update the same document simultaneously, Mongoose can use the version number to detect conflicts and prevent data loss.

Why to use async and await?

Here, result is a Promise, and since the save() operation is asynchronous, console.log(result) is called before the Promise is resolved. That's why you see Promise { <pending> } as the output if you do not use async-await.

How to Fix It

To correctly handle the asynchronous operation and see the result of the save() method, you should use await or handle the Promise using .then().

Example2:

Write a node.js script to enter more than one document to the collection after establishing a connection using mongoose. Raise exception and catch it, if any problem occurs. Print a proper message for error handling.

Insert by creating an Instance

```
const mg=require("mongoose")
mg.connect("mongodb://127.0.0.1:27017/test").then(()=>{console.log("success")}).catch((err)
=>{console.error(err)});
const mySchema=new mg.Schema(
  {
    name:{
      type:String,
      required:true
    },
    Surname:String,
    age:Number,
    active:Boolean,
    date:{
      type:Date,
```

```
        default:new Date()
    }
  }
);
const person=new mg.model("person",mySchema)
const createDoc=async()=>
  {
    try{
    const personData=new person(
      {
        name:"test",
        Surname:"test1",
        age:33,
        active:true
      }
    )
    const personData1=new person(
      {
        name:"hi",
        Surname:"hi1",
        age:30,
        active:true
      }
    )
      const personData2=new person(
        {
          name:"hello",
          Surname:"hello1",
          age:37,
          active:true
        }
      )
      const personData3=new person(
        {
          name:"hello",
          Surname:"hello11",
          age:37,
```

```
            active:true
          }
        )
        const result= await person.insertMany ([personData,personData1,personData2,
personData3])
        console.log(result)
    }
        catch(err)
        {
          console.log("problem");
        }
  }
createDoc();
```

OR

Insert by creating an array of objects

```
const mg=require("mongoose")
mg.connect("mongodb://127.0.0.1:27017/test").then(()=>{console.log("success")}).catch((err)
=>{console.error(err)});
const mySchema=new mg.Schema(
  {
    name:{
      type:String,
      required:true
    },
    Surname:String,
    age:Number,
    active:Boolean,
    date:{
      type:Date,
      default:new Date()
    }
  }
);
const person=new mg.model("person",mySchema)
```

```
const createDoc=async()=>
  {
    try{

    const personData1=[
      {
        name:"test",
        Surname:"test1",
        age:33,
        active:true
      },
      {
        name:"hi",
        Surname:"hi1",
        age:30,
        active:true
      },
      {
          name:"hello",
          Surname:"hello1",
          age:37,
          active:true
        },
        {
          name:"hello",
          Surname:"hello11",
          age:37,
          active:true
        }]
      const result= await person.insertMany(personData1)
       console.log(result)
    }
       catch(err){console.log("problem");}
  }
createDoc();
```

updateOne, findByIdAndUpdate, and findByIdAndDelete

The methods updateOne, findByIdAndUpdate, and findByIdAndDelete in Mongoose are used for different operations on MongoDB documents. Below is an explanation of each, their differences, and when to use them.

1. updateOne

- Updates a single document that matches a given query.

```
Model.updateOne(filter, update, options, callback)
```

- **Parameters:**
  - filter: An object that specifies the criteria to find the document to update.
  - update: An object containing the fields to update.
  - options (optional): An object containing options like:
    - upsert: If true, creates a new document if no document matches the filter. Default is false.
    - runValidators: If true, runs schema validators on the update operation. Useful for maintaining data integrity.
  - callback (optional): A function to execute once the operation completes.
- **Use Cases:**
  - When you want to update a specific document based on a custom query (not just _id).
  - When you need to ensure that only one document is updated, even if multiple documents match the query.

2. findByIdAndUpdate

- Finds a document by its _id and updates it.

```
Model.findByIdAndUpdate(id, update, options, callback)
```

- **Parameters:**
  - id: The _id of the document to find and update.
  - update: An object containing the fields to update.
  - options (optional): An object containing options like:
    - new: If true, returns the updated document instead of the original. Default is false.
    - runValidators: If true, runs schema validators on the update operation.
    - upsert: If true, creates a new document if no document matches the _id. Default is false.
  - callback (optional): A function to execute once the operation completes.
- **Use Cases:**
  - When you need to update a document by its unique _id.
  - When you want to retrieve the updated document immediately after the update.

3. findByIdAndDelete

- Finds a document by its _id and deletes it.

```
Model.findByIdAndDelete(id, options, callback)
```

- **Parameters:**
  - id: The _id of the document to find and delete.
  - options (optional): You can pass an options object, though it's rarely needed for deletion.
  - callback (optional): A function to execute once the operation completes.
- **Use Cases:**
  - When you need to delete a document by its _id.
  - When you want to retrieve the document being deleted for confirmation.

4. findOne

- Finds the **first document** that matches a given query.

```
Model.findOne(filter, projection, options, callback)
```

Parameters:

- **filter:** Specifies the search condition.
- **projection (optional):** Specifies the fields to return.
- **options (optional):** Specifies additional options.
- **callback (optional):** Function executed after the operation.

Use Cases:

- Retrieve the first matching document.
- Search using a field such as **name**, **email**, or **courseName**.

Summary

- **updateOne:** Updates the first document matching a filter. Useful for more complex queries.
- **findByIdAndUpdate:** Finds a document by _id and updates it. Use this when you know the _id of the document you want to update.
- **findByIdAndDelete:** Finds a document by _id and deletes it. Use this when you want to remove a document and potentially get the deleted document back.

Each method has its own specific use cases and should be chosen based on the operation you need to perform.

Example:

```
const mg=require("mongoose");
mg.connect("mongodb://127.0.0.1:27017/lju1");
const personSchema=new mg.Schema({
   name:String,
   age:Number,
   active:Boolean
```

```
});
const Person=mg.model("Person",personSchema);
const performOperations=async()=>{
   try{

      //Insert a document
      const personData=new Person({name:"test",age:25,active:true});
      await personData.save();

      //Update one document
      const result=await
Person.updateOne({name:"test"},{$set:{age:34,active:false}},{upsert:true});
      console.log("Update Result:",result);

      //Find one document
      const person=await Person.findOne({name:"test"});
      console.log("Person:",person);
      console.log("Person ID:",person._id);

      //Update using _id
      const updatedPerson=await
Person.findByIdAndUpdate(person._id,{name:"LJU",age:28,active:true},{new:true});
      console.log("Updated Person:",updatedPerson);

      //Delete using _id
      const deletedPerson=await Person.findByIdAndDelete(person._id);
      if(deletedPerson)
         console.log("Deleted Person:",deletedPerson);
      else
         console.log("Person not found");

   }
   catch(err){console.error(err);}
};
performOperations();
```

Mongoose Schema validation

- ✓ Mongoose Schema validation is a powerful feature that allows you to define rules for your data and ensure that documents saved to the MongoDB database meet these requirements. Validation is performed before saving a document and can help maintain **data integrity and consistency.**
- ✓ Schema validation lets you create validation rules for your fields, such as allowed data types and value ranges.
- ✓ MongoDB uses a flexible schema model, which means that documents in a collection do not need to have the same fields or data types by default.
- ✓ Schema validation is most useful for an established application where you have a good sense of how to organize your data.

When MongoDB Checks Validation

- ✓ When you create a new collection with schema validation, MongoDB checks validation during updates and inserts in that collection.
- ✓ When you add validation to an existing, non-empty collection:
- ✓ Newly inserted documents are checked for validation.
- ✓ Documents already existing in your collection are not checked for validation until they are modified.

What Happens When a Document Fails Validation

- ✓ By default, when an insert or update operation would result in an invalid document, MongoDB rejects the operation and does not write the document to the collection.

Built-in Validators

Mongoose provides several built-in validators for common data validation tasks:

1. **required**: Ensures that a field is present.
2. **min and max**: For number fields, they set minimum and maximum values.
3. **enum**: Ensures that the value of a field is one of a specified set of values.
4. **match**: For string fields, it ensures that the value matches a specified regular expression.
5. **minlength and maxlength**: For string fields, they ensure the length is within a certain range.
6. **trim:** Automatically removes leading and trailing whitespace from a string.
7. **uppercase**: Converts the string to uppercase before saving it to the database.
8. **lowercase**: Converts the string to lowercase before saving it to the database.

9. **default:** Assigns a default value to a field if no value is provided when the document is created.
10. **validate:** Allows for custom validation logic using a function. The function returns true if the value is valid and false otherwise. You can also return a promise for asynchronous validation.

Example

You are developing a MongoDB-based application using Mongoose. You need to define a userSchema that includes various validation rules to ensure data integrity and consistency.

1. **Define a Mongoose schema called userSchema with the following fields and validation requirements:**
    - **username:**
        - **Required and must be between 4 and 20 characters long.**
        - **Must start with letters and end with digits.**
        - **Should be trimmed of any leading or trailing spaces.**
        - **Should be converted to uppercase before saving.**
    - **email:**
        - **Required, must be unique across the collection.**
        - **Must follow the standard email format.**
    - **age:**
        - **Must be a number between 18 and 65.**
    - **role:**
        - **Must be either 'user' or 'admin'.**
        - **Should default to 'user' if not provided.**

```
const mg = require('mongoose');

mg.connect("mongodb://127.0.0.1:27017/validations").then(()=>{console.log("success")}).catch((err)=>{console.error(err)});
const userSchema = new mg.Schema({
  username: {
    type: String,
    required: [true, 'Username is required'], // Custom error message
    minlength: [4, 'Username must be at least 4 characters long'],
    maxlength: [20, 'Username cannot be more than 20 characters long'],
    match:[/^[A-Za-z]+[0-9]+$/,"Must start with letters and end with digits"],
```

```
    trim:true,
    uppercase:true
  },
  email: {
    type: String,
    unique: [true, 'email already exists'],
    required: [true, 'Email is required'],
    match: [/\S+@\S+\.\S+/, 'Please enter a valid email address']
  },
  age: {
    type: Number,
    min: [18, 'Age must be at least 18'],
    max: [65, 'Age must be less than 65']
  },
  role: {
    type: String,
    enum: ['user', 'admin'], // Value must be either 'user' or 'admin'
    default: 'user'
  }
});

const User = mg.model('User', userSchema);
const createDoc=async()=>
   {
      try{
         const newUser = new User({
            username: '  sfga34 ',
            email: ' a1e@xample.com',
            age: 25,
            role: 'user'
          });
         const result= await newUser.save(); //for single data record
         console.log(result);
      }
      catch(err)
      {
         console.log("Error Occured" + err);
      }
   }
   createDoc();
```

Validator

The validator package is a popular library in Node.js that provides a variety of string validation and sanitization functions. When used with Mongoose, it allows you to apply these validations to your schema fields to ensure that the data being stored in MongoDB is valid and meets specific criteria.

Installation of validator module:

```
npm install validator
```

Feature of validator module:

- It is easy to get started and easy to use.
- It is a widely used and popular module for validation.
- Simple functions for validation like isEmail(),isAlphanumeric() etc.

Example:

You are developing a MongoDB application using Mongoose and need to enforce specific validation rules on the email and product fields in your userSchema.

1. **Define a Mongoose schema userSchema with the following fields:**
   - **email:**
     - **The field is required and must be unique in the database.**
     - **It should be validated to ensure it contains a valid email address format.**
     - **If the provided email is invalid, the error message should indicate that the email address is not valid.**
   - **product:**
     - **The field is required.**
     - **It should only allow alphanumeric characters (i.e., letters and numbers).**
     - **If the product field contains invalid characters, the error message should indicate that it is not a valid alphanumeric code.**
2. **Write an asynchronous function createDoc that creates and saves a document with random data.**

```
const mg = require('mongoose');
const validator = require('validator');
mg.connect("mongodb://127.0.0.1:27017/validations").then(()=>{console.log("success")}).c
atch((err)=>{console.error(err)});
const userSchema = new mg.Schema({
  email: {
    type: String,
    required: true,
    unique: true,
    validate: [validator.isEmail,`This is not a valid email!`]
  },
```

```
  product: {
    type: String,
    required: true,
    validate: [validator.isAlphanumeric,`This is not a valid alphanumeric code!`]
    }

});

const User = mg.model('User', userSchema);
const createDoc=async()=>
   {
      try{
         const newUser = new User({
            email: '1agecc@gmail.com',
            product:"fhxvf111"
          });
         const result= await newUser.save(); //for single data record
         console.log(result);
      }
      catch(err)
      {
         console.log("Error Occured" + err);
      }
   }
   createDoc();
```

Example

Write a node.js script to define a schema having fields like name, age, gender, email.

Apply following validations:

(1) Name field must remove leading/trailing spaces, minimum and maximum length should be 3 & 10 respectively, and name should be stored in lowercase

(2) Age must accept value greater than 0.

(3) Perform Email ID validation on Email field.

(4) Gender must accept values in uppercase only and allowed values are “MALE” & “FEMALE” only.

```
const mongoose = require('mongoose');
const validator = require('validator');

// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/validations")
 .then(() => console.log("Connected to MongoDB"))
 .catch((err) => console.error("Connection error:", err));

// Define the schema
const userSchema = new mongoose.Schema({
 name: {
  type: String,
  required: [true, 'Name is required'],
  minlength: [3, 'Name must be at least 3 characters long'],
  maxlength: [10, 'Name cannot be more than 10 characters long'],
  trim: true,
  lowercase:true
 },
 age: {
  type: Number,
  required: [true, 'Age is required'],
  min: [1, 'Age must be greater than 0']
 },
 email: {
  type: String,
  required: [true, 'Email is required'],
  unique: true,
  validate: [validator.isEmail,`This is not a valid email!`]
 },
```

```
  gender: {
    type: String,
    required: [true, 'Gender is required'],
    uppercase: true, // Converts the gender to uppercase before saving
    enum:['MALE', 'FEMALE']
  }
});

// Create the model
const User = mongoose.model('User', userSchema);

// Function to create a document
const createDoc = async () => {
  try {
    const newUser = new User({
      name: '  LjUuser  ', // Leading and trailing spaces will be trimmed, and name will be stored in lowercase
      age: 25,
      email: 'lju@example.com',
      gender: 'male' // This will be converted to 'MALE'
    });

    const result = await newUser.save(); // Save the document
    console.log(result);
  } catch (err) {
    console.error("Error occurred:", err.message);
  }
};

// Call the function to create a document
createDoc();
```

Queries
Example

Write a Node.js script using Mongoose to perform the following operations on the **movies** collection.

1. Insert multiple movie documents.
2. Display all movies having a rating greater than **8.5**.
3. Display the **title** and **rating** of the movie having the **second highest rating**.
4. Increase the rating of all **Action** movies by **0.2**.
5. Count the total number of **Hindi** movies.
6. Delete the movie having the title **"Jawan"**.

```
const mg = require("mongoose");

mg.connect("mongodb://127.0.0.1:27017/lju")
.then(()=>console.log("Connected"))
.catch(err=>console.log(err));

const movieSchema = new mg.Schema({
   title:String,
   director:String,
   genre:String,
   rating:Number,
   releaseYear:Number,
   language:String,
   available:Boolean
});

const Movie = mg.model("Movie", movieSchema);

const movies=[
{title:"3 Idiots", director:"Rajkumar Hirani", genre:"Comedy", rating:9.2, releaseYear:2009,
language:"Hindi"},
{title:"KGF Chapter 2", director:"Prashanth Neel", genre:"Action", rating:8.8, releaseYear:2022,
language:"Kannada"},
{title:"Dangal", director:"Nitesh Tiwari", genre:"Drama", rating:8.9, releaseYear:2016,
language:"Hindi"},
{title:"Baahubali", director:"S. S. Rajamouli", genre:"Action", rating:8.7, releaseYear:2015,
language:"Telugu"},
```

```
{title:"Jawan", director:"Atlee", genre:"Action", rating:7.8, releaseYear:2023,
language:"Hindi"},
{title:"Drishyam", director:"Nishikant Kamat", genre:"Thriller", rating:8.4, releaseYear:2015,
language:"Hindi"},
{title:"Pushpa", director:"Sukumar", genre:"Action", rating:8.1, releaseYear:2021,
language:"Telugu"}
];

const performOperations=async()=>{

try{
await Movie.insertMany(movies);
console.log(await Movie.find({rating:{$gt:8.5}}));
console.log(
await Movie.find({},{title:1,rating:1,_id:0}).sort({rating:-1}).skip(1).limit(1)
);
console.log(
await Movie.updateMany(
{genre:"Action"},
{$inc:{rating:0.2}}
)
);

console.log(
await Movie.countDocuments({language:"Hindi"})
);

console.log(
await Movie.deleteOne({title:"Jawan"})
);

}
catch(err){
console.log(err);
}
}
performOperations();
```

Example

Write a Node.js script using Mongoose to perform the following operations on the **courses** collection.

1. Insert multiple course documents.
2. Display the course having the highest fees.
3. Find the course named **"MERN Stack Development"**.
4. Update the fees to **₹15,000** and duration to **5 months** for the course named **"MERN Stack Development"** using **findByIdAndUpdate**.
5. Display all **Online** courses having fees less than **₹20,000**.
6. Display all **active** courses whose duration is greater than **4 months**, but exclude courses that are **Online** or belong to the **Cloud** category.
7. Delete the course named **"MERN Stack Development"** using its **_id**.

```
const mg = require("mongoose");

mg.connect("mongodb://127.0.0.1:27017/lju")
.then(() => console.log("Connected"))
.catch((err) => console.log(err));

const courseSchema = new mg.Schema({
   courseName: String,
   instructor: String,
   duration: Number,
   fees: Number,
   mode: String,
   category: String,
   active: Boolean
});

const Course = mg.model("Course", courseSchema);

const courses = [
{courseName:"MERN Stack Development", instructor:"Niharika Sen", duration:6, fees:18000,
mode:"Offline", category:"Web Development", active:true},
{courseName:"Python Programming", instructor:"Rahul Shah", duration:4, fees:12000,
mode:"Online", category:"Programming", active:true},
{courseName:"Data Science", instructor:"Priya Patel", duration:8, fees:25000, mode:"Offline",
category:"Data Analytics", active:true},
```

```
{courseName:"Machine Learning", instructor:"Amit Joshi", duration:7, fees:22000,
mode:"Online", category:"Artificial Intelligence", active:false},
{courseName:"Java Full Stack", instructor:"Neha Mehta", duration:6, fees:20000,
mode:"Offline", category:"Web Development", active:true},
{courseName:"UI/UX Design", instructor:"Karan Desai", duration:3, fees:10000,
mode:"Online", category:"Design", active:true},
{courseName:"Cloud Computing", instructor:"Riya Sharma", duration:5, fees:16000,
mode:"Offline", category:"Cloud", active:false}
];

const performOperations = async () => {

   try {

      const result = [];
      // 1. Insert multiple course documents
      result.push(await Course.insertMany(courses));
      // 2. Display the course having the highest fees
      result.push(await Course.find().sort({ fees: -1 }).limit(1));
      // 3. Find the course named "MERN Stack Development"
      const course = await Course.findOne({courseName: "MERN Stack Development"});
      result.push(course);

      // 4. Update fees and duration
      result.push(
    await Course.findByIdAndUpdate(course._id,{$set:{fees: 15000,duration: 5}},{ new: true }));

      // 5. Display all Online courses having fees less than 20000
      result.push(await Course.find({mode: "Online", fees: { $lt: 20000 }}));

      // 6. Display all active courses whose duration is greater than 4 months, but exclude
Online and Cloud courses
      result.push(
         await Course.find({
            $and: [{ active: true }, { duration: { $gt: 4 } }],
            $nor: [{ mode: "Online" },{ category: "Cloud" }]
         }) );
```

```
    // 7. Delete the course using its _id
    result.push(
      await Course.findByIdAndDelete(course._id)
    );
    console.log(result);
  } catch (err) {console.log(err);}
};
performOperations();
```

Connectivity of MongoDB with ExpressJS/NodeJS using HTML

Install express if not already installed using below command.

```
npm install express
```

Example:
Create html file which contains a form having username and password and insert data entered by user in collection named "data1".

task.js

```
var expr=require("express")
var app=expr()
const mg=require("mongoose")

mg.connect("mongodb://127.0.0.1:27017/login")
.then(()=>{console.log("Successful")})
.catch((err)=>{console.error(err)})

mg.pluralize(null)
const myschema=new mg.Schema({
      uname:{type:String, required:true},
      password: {type:String, required:true} })
const person =new mg.model("data1", myschema)

app.use(expr.static(__dirname,{index:"form.html"}))

app.get("/process_get",async (req,res)=>{
     const personData=new person({
     uname:req.query.uname,
     password:req.query.pwd
})
await personData.save()
res.send("Record inserted")
})
app.listen(6000)
```

form.html

```
<html>
  <form action="/process_get" method="get">
    Username: <input type="text" name="uname"/>
  </br> Password: <input type="password" name="pwd"/>
  </br> <input type="submit"/>
  </form>
</html>
```

To run

In terminal node task.js
In browser localhost:6000

How It Works:

1. When the server is running and a user navigates to the root URL (e.g., http://localhost:6000), the form.html page is served.
2. The user fills in the username and password fields in the form and submits it.
3. The form data is sent to the server via a GET request to the /process_get endpoint.
4. The server extracts the data from the query parameters, creates a new document using the person model, and saves it to the data1 collection in the login database.
5. Upon successful insertion, the server responds with the message "Record inserted".

Running the Application:

- **Start the server:** Run node task.js in the terminal. Ensure that MongoDB is running on your machine.
- **Access the form:** Open a web browser and navigate to http://localhost:6000. You will see the form where you can enter a username and password.
- **Submit the form:** After submitting, if everything is set up correctly, you'll see the message "Record inserted," and the data will be saved to the MongoDB database.

React-express/node-mongodb

Frontend

Create a React Client

```
npm create vite@latest
```

Next, install Axios. This package will enable you to send HTTP requests to your backend sxpress.js server to store data in your MongoDB database.

```
npm install axios
```

Axios

Fetching User Data: If you want to retrieve user information from the server:

```
axios.get('http://localhost:5000/users')
  .then(response => {
    // Process the response data
  });
```

Creating a New User: If you want to create a new user by sending data to the server:

```
axios.post('http://localhost:5000/signup', { username: 'newUser' })
  .then(response => {
    // Handle success
  });
```

```
react-connect/
|
├── client/                 # React (Vite) Frontend
|   |
|   ├── node_modules/
|   ├── public/
|   |
|   ├── src/
|   |   ├── Client.jsx      # Client component
|   |   ├── App.jsx
|   |   ├── main.jsx
|   |   └── index.css
|   |
|   ├── package.json
|   ├── vite.config.js
|   └── index.html
|
├── server/                 # Express Backend
|   |
```

```
|   ├── node_modules/
|   ├── server.js          # Express server
|   └── package.json
```

client/src/Client.js

```
import { useState } from 'react';
import axios from 'axios';

function Client() {
 const [username, setUsername] = useState('');

 const handleSignup = async(e) => {
  e.preventDefault();
  try {
   await axios.post('http://localhost:5000/signup', { username });
   alert('Welcome ' + username);
   setUsername('');
  } catch (error) { console.error('Error signing up:', error); alert('An error occurred.'); }
 };

 return (
  <div>
   <h1>Sign Up</h1>
   <form onSubmit={handleSignup}>
    <input type="text" placeholder="Username" value={username} onChange={(e) =>
setUsername(e.target.value)}/>
    <button type="submit">Sign Up</button>
    <h1>{username}</h1>
   </form>
  </div>
 );
}

export default Client;
```

To run

```
cd client
npm run dev
```

Backend

Dependencies:

- **express**: A web framework for Node.js used to create the server. **(npm i express)**
- **mongoose**: A MongoDB object modeling tool designed to work in an asynchronous environment. **(npm i mongoose)**
- **cors**: A middleware to allow cross-origin requests. **(npm i cors)**

Commands for backend

```
mkdir server
cd server
npm init -y
npm i express mongoose cors
```

The cors middleware

It is used to enable Cross-Origin Resource Sharing (CORS). CORS is a security feature implemented by web browsers that restricts how resources on a web page can be requested from a different domain than the one that served the web page.

Why CORS is Important:

When you're developing a frontend (e.g., React) and backend (e.g., Express) separately, they often run on different domains or ports during development (e.g., React on localhost:3000 and Express on localhost:5000). This difference can trigger the browser's same-origin policy, which blocks requests from the frontend to the backend.

CORS is necessary to allow the frontend to communicate with the backend. By using the cors middleware, you tell the Express server to include specific headers in its responses, enabling the frontend to bypass the same-origin policy.

How CORS is Used in the Code:

```
app.use(cors());
```

This line adds the cors middleware with the default settings, which allows requests from any origin (i.e., all domains). This is particularly useful during development, but in a production environment, you might want to restrict it to specific domains.

express.json():

express.json() parses incoming JSON data sent by the client and stores it in **req.body** so it can be accessed inside Express routes.

Body Parsing: The express.urlencoded({ extended: false }) middleware only parses URL-encoded bodies (typically from forms). Since your React application likely sends data as JSON, the server does not understand it, leading to req.body being empty.
Add express.json() to parse incoming JSON requests.

server/server.js

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect('mongodb://127.0.0.1:27017/reactconnect');

const UserSchema = new mongoose.Schema({
  username:String
});

const User = new mongoose.model('User', UserSchema);

app.post('/signup', async (req, res) => {
  try {
    const { username} = req.body;
    const newUser = new User({ username});
    await newUser.save();
    res.send();
  } catch (error) {
    res.send(error);
  }
});app.listen(5000);
```

To run

```
server> node server.js
```

How It Works:

1. A user fills in their username on the frontend and submits the form.
2. The React app sends a POST request to the Express server with the username.

3. The Express server receives the request, creates a new user document in the MongoDB database, and saves the username.
4. If successful, the React app displays a welcome message with the username and clears the input field.

Running the Application:

- **Frontend:**
  - Start the React app by running npm run dev in the terminal. This will start the development server for the React app.
- **Backend:**
  - Start the Express server by running node server.js in the terminal. This will connect to MongoDB and start listening on port 5000 for incoming requests.

The frontend and backend need to be running simultaneously, with the React app on the default port (5173) and the Express server on port 5000.